half the output bits, i.e., the second ciphertext looks statistically independent of the first one. This is an important property to keep in mind when dealing with block ciphers. We demonstrate this behavior with the following simple example.

Example 3.1. Let's assume a small block cipher with a block length of 8 bits. Encryption of two plaintexts x_1 and x_2, which differ only by one bit, should roughly result in something as shown in Fig. 3.2.

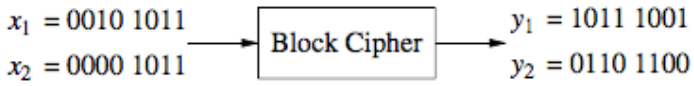


Fig. 3.2 Principle of diffusion of a block cipher

Note that modern block ciphers have block lengths of 64 or 128 bit but they show exactly the same behavior if one input bit is flipped.

◇

3.2 Overview of the DES Algorithm

DES is a cipher which encrypts blocks of length of 64 bits with a key of size of 56 bits (Fig. 3.3).

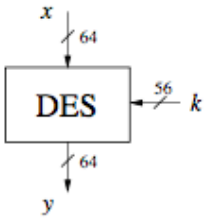


Fig. 3.3 DES block cipher

DES is a symmetric cipher, i.e., the same same key is used for encryption and decryption. DES is, like virtually all modern block ciphers, an iterative algorithm. For each block of plaintext, encryption is handled in 16 rounds which all perform the identical operation. Figure 3.4 shows the round structure of DES. In every round a different subkey is used and all subkeys k_i are derived from the main key k.

Let's now have a more detailed view on the internals of DES, as shown in Fig. 3.5.The structure in the figure is called a *Feistel network*. It can lead to very strong ciphers if carefully designed. Feistel networks are used in many, but certainly not in all, modern block ciphers. (In fact, AES is not a Feistel cipher.) In addition to its potential cryptographic strength, one advantage of Feistel networks is that encryption and decryption are almost the same operation. Decryption requires

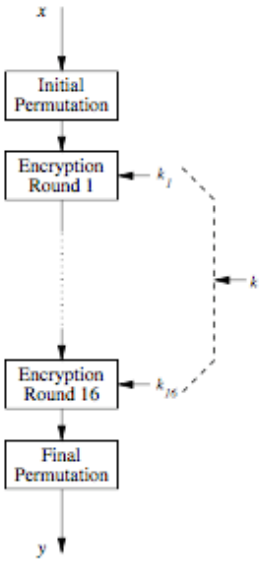


Fig. 3.4 Iterative structure of DES

only a reversed key schedule, which is an advantage in software and hardware implementations. We discuss the Feistel network in the following.

After the initial bitwise permutation IP of a 64-bit plaintext x, the plaintext is split into two halves L_0 and R_0. These two 32-bit halves are the input to the Feistel network, which consists of 16 rounds. The right half R_i is fed into the function f. The output of the f function is XORed (as usually denoted by the symbol $\oplus$) with the left 32-bit half L_i. Finally, the right and left half are swapped. This process repeats in the next round and can be expressed as:

$$L_i = R_{i-1},$$
$$R_i = L_{i-1} \oplus f(R_{i-1}, k_i)$$

where i= 1,...,16. After round 16, the 32-bit halves L_{16} and R_{16} are swapped again, and the final permutation IP^{-1} is the last operation of DES. As the notation suggests, the final permutation IP^{-1} is the inverse of the initial permutation IP. In each round, a round key k_i is derived from the main 56-bit key using what is called the key schedule.

It is crucial to note that the Feistel structure really only encrypts (decrypts) half of the input bits per each round, namely the left half of the input. The right half is copied to the next round unchanged. In particular, the right half is *not encrypted* with the f function. In order to get a better understanding of the working of Feistel cipher, the following interpretation is helpful: Think of the f function as a pseudorandom generator with the two input parameters R_{i-1} and k_i. The output of the pseudorandom generator is then used to encrypt the left half L_{i-1} with an XOR operation. As we saw in Chap. 2, if the output of the f function is not predictable for an attacker, this results in a strong encryption method.

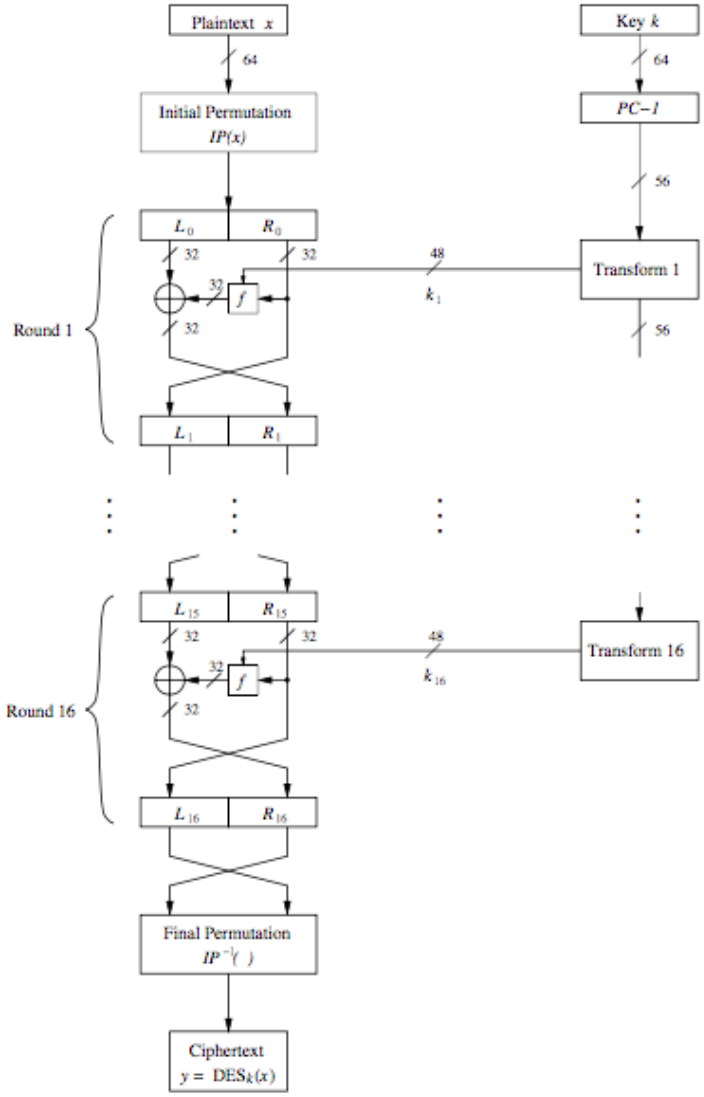


Fig. 3.5 The Feistel structure of DES

The two aforementioned basic properties of ciphers, i.e., confusion and diffusion, are realized within the f-function. In order to thwart advanced analytical attacks, the f-function must be designed extremely carefully. Once the f-function has been designed securely, the security of a Feistel cipher increases with the number of key bits used and the number of rounds.

Before we discuss all components of DES in detail, here is an algebraic description of the Feistel network for the mathematically inclined reader. The Feistel structure of each round bijectively maps a block of 64 input bits to 64 output bits (i.e., every possible input is mapped uniquely to exactly one output, and vice versa). This mapping remains bijective for some arbitrary function f, i.e., even if the embedded function f is not bijective itself. In the case of DES, the function f is in fact a sur-

jective (many-to-one) mapping. It uses nonlinear building blocks and maps 32 input bits to 32 output bits using a 48-bit round key k_i, with $1 \leq i \leq 16$.

3.3 Internal Structure of DES

The structure of DES as depicted in Fig. 3.5 shows the internal functions which we will discuss in this section. The building blocks are the initial and final permutation, the actual DES rounds with its core, the f-function, and the key schedule.

3.3.1 Initial and Final Permutation

As shown in Figs. 3.6 and 3.7, the *initial permutation IP* and the *final permutation* IP^{-1} are bitwise permutations. A bitwise permutation can be viewed as simple crosswiring. Interestingly, permutations can be very easily implemented in hardware but are not particularly fast in software. Note that both permutations do not increase the security of DES at all. The exact rationale for the existence of these two permutations is not known, but it seems likely that their original purpose was to arrange the plaintext, ciphertext and bits in a bytewise manner to make data fetches easier for 8-bit data busses, which were the state-of-the-art register size in the early 1970s.

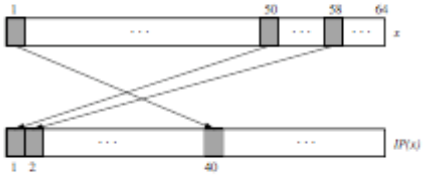


Fig. 3.6 Examples for the bit swaps of the initial permutation

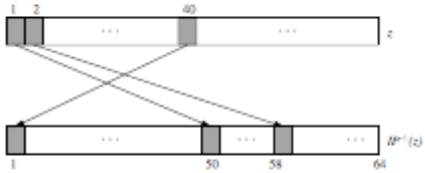


Fig. 3.7 Examples for the bit swaps of the final permutation

The details of the transformation *IP* are given in Fig. 3.8. This table, like all other tables in this chapter, should be read from left to right, top to bottom. The table indicates that input bit 58 is mapped to output position 1, input bit 50 is mapped to

the second output position, and so forth. The final permutation IP^{-1} performs the inverse operation of IP as shown in Fig. 3.9.

IP							
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Fig. 3.8 Initial permutation IP

IP^{-1}							
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Fig. 3.9 Final permutation IP^{-1}

3.3.2 The f-Function

As mentioned earlier, the f-function plays a crucial role for the security of DES. In round i it takes the right half R_{i-1} of the output of the previous round and the current round key k_i as input. The output of the f-function is used as an XOR-mask for encrypting the left half input bits L_{i-1}.

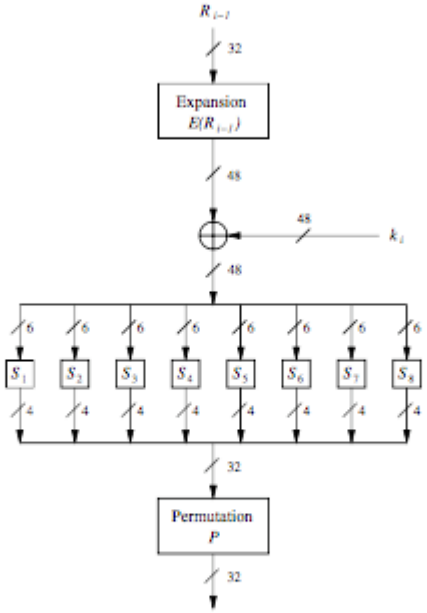


Fig. 3.10 Block diagram of the f-function

The structure of the f-function is shown in Fig. 3.10. First, the 32-bit input is expanded to 48 bits by partitioning the input into eight 4-bit blocks and by expanding each block to 6 bits. This happens in the E-box, which is a special type of permutation. The first block consists of the bits $(1,2,3,4)$, the second one of $(5,6,7,8)$, etc. The expansion to six bits can be seen in Fig. 3.11.

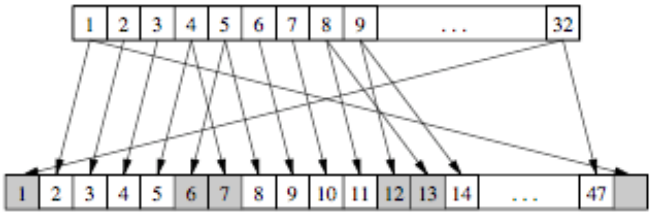


Fig. 3.11 Examples for the bit swaps of the expansion function E

As can be seen from the Table 3.1, exactly 16 of the 32 input bits appear twice in the output. However, an input bit never appears twice in the same 6-bit output block. The expansion box increases the diffusion behavior of DES since certain input bits influence two different output locations.

Table 3.1 Expansion permutation E

E					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Next, the 48-bit result of the expansion is XORed with the round key k_i, and the eight 6-bit blocks are fed into eight different *substition boxes*, which are often referred to as *S-boxes*. Each S-box is a lookup table that maps a 6-bit input to a 4-bit output. Larger tables would have been cryptographically better, but they also become much larger; eight 4-by-6 tables were probably close the maximum size which could be fit on a single integrated circuit in 1974. Each S-box contains $2^6 = 64$ entries, which are typically represented by a table with 16 columns and 4 rows. Each entry is a 4-bit value. All S-boxes are listed in Tables 3.2 to 3.9. Note that all S-boxes are different. The tables are to be read as indicated in Fig. 3.12: the most significant bit (MSB) and the least significant bit (LSB) of each 6-bit input select the row of the table, while the four inner bits select the column. The integers 0,1,...,15 of each entry in the table represent the decimal notation of a 4-bit value.

Example 3.2. The S-box input $b = (100101)_2$ indicates the row $11_2 = 3$ (i.e., fourth row, numbering starts with 00_2) and the column $0010_2 = 2$ (i.e., the third column). If the input b is fed into S-box 1, the output is $S_1(37 = 100101_2) = 8 = 1000_2$.

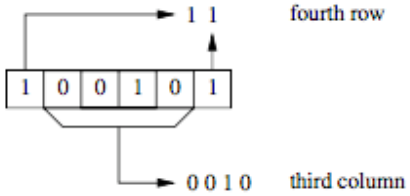


Fig. 3.12 Example of the decoding of the input 100101_2 by S-box 1

◇

Table 3.2 S-box S_1

S_1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	01	10	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

Table 3.3 S-box S_2

S_2	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	15	01	08	14	06	11	03	04	09	07	02	13	12	00	05	10
1	03	13	04	07	15	02	08	14	12	00	01	10	06	09	11	05
2	00	14	07	11	10	04	13	01	05	08	12	06	09	03	02	15
3	13	08	10	01	03	15	04	02	11	06	07	12	00	05	14	09

Table 3.4 S-box S_3

S_3	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	10	00	09	14	06	03	15	05	01	13	12	07	11	04	02	08
1	13	07	00	09	03	04	06	10	02	08	05	14	12	11	15	01
2	13	06	04	09	08	15	03	00	11	01	02	12	05	10	14	07
3	01	10	13	00	06	09	08	07	04	15	14	03	11	05	02	12

The S-boxes are the core of DES in terms of cryptographic strength. They are the only nonlinear element in the algorithm and provide confusion. Even though the entire specification of DES was released by NBS/NIST in 1977, the motivation for the choice of the S-box tables was never completely revealed. This often gave rise

Table 3.5 S-box S_4

S_4	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	07	13	14	03	00	06	09	10	01	02	08	05	11	12	04	15
1	13	08	11	05	06	15	00	03	04	07	02	12	01	10	14	09
2	10	06	09	00	12	11	07	13	15	01	03	14	05	02	08	04
3	03	15	00	06	10	01	13	08	09	04	05	11	12	07	02	14

Table 3.6 S-box S_5

S_5	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	02	12	04	01	07	10	11	06	08	05	03	15	13	00	14	09
1	14	11	02	12	04	07	13	01	05	00	15	10	03	09	08	06
2	04	02	01	11	10	13	07	08	15	09	12	05	06	03	00	14
3	11	08	12	07	01	14	02	13	06	15	00	09	10	04	05	03

Table 3.7 S-box S_6

S_6	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	01	10	15	09	02	06	08	00	13	03	04	14	07	05	11
1	10	15	04	02	07	12	09	05	06	01	13	14	00	11	03	08
2	09	14	15	05	02	08	12	03	07	00	04	10	01	13	11	06
3	04	03	02	12	09	05	15	10	11	14	01	07	06	00	08	13

Table 3.8 S-box S_7

S_7	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	04	11	02	14	15	00	08	13	03	12	09	07	05	10	06	01
1	13	00	11	07	04	09	01	10	14	03	05	12	02	15	08	06
2	01	04	11	13	12	03	07	14	10	15	06	08	00	05	09	02
3	06	11	13	08	01	04	10	07	09	05	00	15	14	02	03	12

Table 3.9 S-box S_8

S_8	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	13	02	08	04	06	15	11	01	10	09	03	14	05	00	12	07
1	01	15	13	08	10	03	07	04	12	05	06	11	00	14	09	02
2	07	11	04	01	09	12	14	02	00	06	10	13	15	03	05	08
3	02	01	14	07	04	10	08	13	15	12	09	00	03	05	06	11

to speculation, in particular with respect to the possible existence of a secret back door or some other intentionally constructed weakness, which could be exploited by the NSA. However, now we know that the S-boxes were designed according to the criteria listed below.

1. Each S-box has six input bits and four output bits.
2. No single output bit should be too close to a linear combination of the input bits.
3. If the lowest and the highest bits of the input are fixed and the four middle bits are varied, each of the possible 4-bit output values must occur exactly once.
4. If two inputs to an S-box differ in exactly one bit, their outputs must differ in at least two bits.

5. If two inputs to an S-box differ in the two middle bits, their outputs must differ in at least two bits.
6. If two inputs to an S-box differ in their first two bits and are identical in their last two bits, the two outputs must be different.
7. For any nonzero 6-bit difference between inputs, no more than 8 of the 32 pairs of inputs exhibiting that difference may result in the same output difference.
8. A collision (zero output difference) at the 32-bit output of the eight S-boxes is only possible for three adjacent S-boxes.

Note that some of these design criteria were not revealed until the 1990s. More information about the issue of the secrecy of the design criteria is found in Sect. 3.5.

The S-boxes are the most crucial elements of DES because they introduce a *nonlinearity* to the cipher, i.e.,

$$S(a) \oplus S(b) \neq S(a \oplus b).$$

Without a nonlinear building block, an attacker could express the DES input and output with a system of linear equations where the key bits are the unknowns. Such systems can easily be solved, a fact that was used in the LFSR attack in Sect. 2.3.2. However, the S-boxes were carefully designed to also thwart advanced mathematical attacks, in particular *differential cryptanalysis*. Interestingly, differential cryptanalysis was first discovered in the research community in 1990. At this point, the IBM team declared that the attack was known to the designers at least 16 years earlier, and that DES was especially designed to withstand differential cryptanalysis.

Finally, the 32-bit output is permuted bitwise according to the P permutation, which is given in Table 3.10. Unlike the initial permutation IP and its inverse IP^{-1}, the permutation P introduces diffusion because the four output bits of each S-box are permuted in such a way that they affect several different S-boxes in the following round. The diffusion caused by the expansion, S-boxes and the permutation P guarantees that every bit at the end of the fifth round is a function of every plaintext bit and every key bit. This behavior is known as the *avalanche effect*.

Table 3.10 The permutation P within the f-function

P							
16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

3.3.3 Key Schedule

The *key schedule* derives 16 round keys k_i, each consisting of 48 bits, from the original 56-bit key. Another term for round key is subkey. First, note that the DES input key is often stated as 64-bit, where every eighth bit is used as an odd parity bit over the preceding seven bits. It is not quite clear why DES was specified that way. In any case, the eight parity bits are **not** actual key bits and do not increase the security. DES is a 56-bit cipher, not a 64-bit one.

As shown in Fig. 3.13, the 64-bit key is first reduced to 56 bits by ignoring every eighth bit, i.e., the parity bits are stripped in the initial $PC-1$ permutation. Again, the parity bits certainly do not increase the key space! The name $PC-1$ stands for "permuted choice one". The exact bit connections that are realized by $PC-1$ are given in Table 3.11.

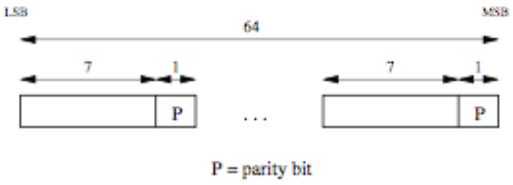


Fig. 3.13 Location of the eight parity bits for a 64-bit input key

Table 3.11 Initial key permutation $PC-1$

$PC-1$							
57	49	41	33	25	17	9	1
58	50	42	34	26	18	10	2
59	51	43	35	27	19	11	3
60	52	44	36	63	55	47	39
31	23	15	7	62	54	46	38
30	22	14	6	61	53	45	37
29	21	13	5	28	20	12	4

The resulting 56-bit key is split into two halves C_0 and D_0, and the actual key schedule starts as shown in Fig. 3.14. The two 28-bit halves are cyclically shifted, i.e., rotated, left by one or two bit positions depending on the round i according to the following rules:

- In rounds $i = 1,2,9,16$, the two halves are rotated left by one bit.
- In the other rounds where $i \neq 1,2,9,16$, the two halves are rotated left by two bits.

Note that the rotations only take place within either the left or the right half. The total number of rotation positions is $4 \cdot 1 + 12 \cdot 2 = 28$. This leads to the interesting property that $C_0 = C_{16}$ and $D_0 = D_{16}$. This is very useful for the decryption key

schedule where the subkeys have to be generated in reversed order, as we will see in Sect. 3.4.

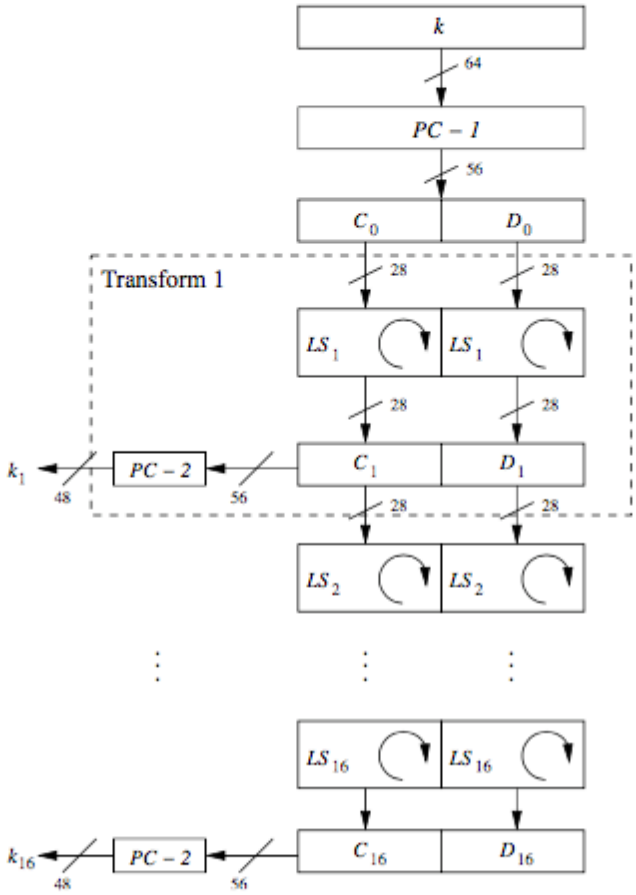


Fig. 3.14 Key schedule for DES encryption

To derive the 48-bit round keys k_i, the two halves are permuted bitwise again with $PC-2$, which stands for "permuted choice 2". $PC-2$ permutes the 56 input bits coming from C_i and D_i and ignores 8 of them. The exact bit-connections of $PC-2$ are given in Table 3.12.

Table 3.12 Round key permutation $PC-2$

$PC-2$							
14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32